Exploring Deep Learning and Machine Learning Approaches for Brain Hemorrhage Detection

Project Phase-II Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

Information Technology

By

Mr. PAKALAPATI ABHINAV VARMA (160922737039)

Mr. MOHD IRFAN UDDIN (160922737003)

Ms. SIMEEN FATIMA JAMEEL (160922737032)

Under the Guidance of

Mrs. BHARGAVI

ASSISTANT PROFESSOR

DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University)

Himayathsagar, Hyderabad,500091

A.Y. 2025-2026

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091

Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-II report entitled Exploring Deep Learning and Machine Learning Approaches for Brain Hemorrhage Detection being submitted by **Mr. Pakalapati Abhinav Varma (160922737039), Mr. Mohd Irfan Uddin (160922737003), Ms. Simeen Fatima Jameel (160922737032)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of bonafide work carried out by them.

Internal Project Guide
Mrs. BHARGAVI
ASSISTANT PROFESSOR

Dr. K. Nagi Reddy
H.O.D, I.T

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

DECLARATION BY THE CANDIDATES

We **Mr. Pakalapati Abhinav Varma (160922737039), Mr. Mohd Irfan Uddin (160922737003), Ms. Simeen Fatima Jameel (160922737032)**, hereby declare that the Project Phase-II report entitled Exploring Deep Learning and Machine Learning Approaches for Brain Hemorrhage Detection is carried out under the guidance of Mrs. Bhargavi, Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology**, **Hyderabad** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Mr. PAKALAPATI ABHINAV VARMA (160922737039)
Mr. MOHD IRFAN UDDIN (160922737003)
Ms. SIMEEN FATIMA JAMEEL (160922737032)

ACKNOWLEDGMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding a number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express my deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Mrs. Bhargavi, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for her guidance and help. Her insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Mr. PAKALAPATI ABHINAV VARMA (160922737039)

Mr. MOHD IRFAN UDDIN (160922737003)

Ms. SIMEEN FATIMA JAMEEL (160922737032)

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department

Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adopt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Outcomes: PP.CO- Project Phase- II

After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program related to the real-world problems.	**Apply**
PP.CO2	Organize and examine different solutions based on economic, technical and functional feasibilities.	**Analyse**
PP.CO3	Provide support and interpret the critics for implementing the better solutions to formulate the robust project.	**Evaluate**
PP.CO4	Design and develop a model using technical and functional provisions to make the formulated solutions.	**Create**
PP.CO5	Compose the project components and apply documentation standards to prepare the project report and present the project work.	**Create**

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.CO1	3	2	-	2	1	-	-	-	-	-	2	3	2
PP.CO2	2	3	2	3	-	-	-	-	-	1	2	2	3
PP.CO3	3	2	3	3	2	2	-	-	-	-	3	3	3
PP.CO4	3	2	3	1	3	2	3	3	3	2	3	3	3
PP.CO5	2	-	3	-	2	1	3	3	2	2	3	3	3
Average	2.6	2.25	2.75	2.25	2.66	1.66	3	3	2.5	1.6	2.6	2.8	2.8

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING	✓	9 INDUSTRY, INNOVATION AND INFRASTRUCTURE		15 LIFE ON LAND	
4 QUALITY EDUCATION		10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

Brain stroke detection is a critical medical process that requires accurate and timely diagnosis to prevent severe complications and reduce mortality rates. Delays in identifying stroke conditions, especially brain hemorrhage, may result in irreversible brain damage. This project, titled *Brain Hemorrhage Detection System based on CT Images using Deep Learning*, presents an intelligent diagnostic support system that improves the efficiency and accuracy of stroke detection using advanced deep learning techniques.

The proposed system is developed using Python as the backend programming language, with Flask serving as a lightweight web framework to manage server-side processing and image handling. The frontend interface is designed using HTML, CSS, and JavaScript, providing a simple and user-friendly platform that enables medical professionals to upload CT scan images and view diagnostic results efficiently.

At the core of the system, a three-layered Convolutional Neural Network (CNN) is implemented to automatically extract relevant features from CT images and detect hemorrhage patterns. The model achieves a training accuracy of 98.00% and a validation accuracy of 97.00%, demonstrating strong predictive capability and reliable generalization performance. The dataset used consists of 2,501 CT images, including 1,551 normal cases and 950 hemorrhage cases, ensuring diversity and robustness in training.

Overall, the integration of deep learning with web-based technologies provides a scalable and effective solution for assisting healthcare professionals in early stroke detection, thereby improving diagnostic speed and patient outcomes.

Keywords — Brain Stroke Detection, Brain Hemorrhage, Deep Learning, Convolutional Neural Network (CNN), CT Images, Medical Image Processing, Artificial Intelligence in Healthcare, Flask Framework, Web-Based Diagnostic System.

CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Vision and Mission of the Institute	iv
Vision and Mission of the Department	v
Program Educational Objectives	vi
Program Outcomes	vii
Program Specific Outcomes	viii
Course Outcomes	ix
Course Articulation Matrix	x
Abstract	xi
Contents	xii
List of Figures	xiv
List of Tables	xv
CHAPTER-1: INTRODUCTION	**1**
1.1 Objective	2
1.2 Problem Formulation	2
1.3 Unique Features of the System	3
CHAPTER-2: LITERATURE SURVEY	**4**
CHAPTER-3: SYSTEM ANALYSIS & REQUIREMENTS	**12**
CHAPTER-4: SYSTEM DESIGN	**19**
4.1 System Architecture	19
4.2 Data Flow Diagram	20
4.3 UML Diagram	22
4.4 Use Case Diagram	22
4.5 Sequence Diagram	23
4.6 Activity Diagram	24
4.7 Class Diagram	26
CHAPTER-5: SOURCE CODE	**27**
CHAPTER-6: RESULT	**34**
CONCLUSION	**39**
REFERENCES	**41**
APPENDIX	**43**

List of Figures

SI.no	Fig.no	Name of the Figure	Page No
1	4.1	System Architecture	20
2	4.2	Data Flow Diagram	22
3	4.3	Use Case Diagram	24
4	4.4	Sequence Diagram	25
5	4.5	Activity Diagram	26
6	4.6	Class Diagram	27
7	6.1	Home Page	34
8	6.2	CNN(Accuracy Curve & Loss Curve)	34
9	6.3	CNN(ROC Curve & Confusion Matrix)	35
10	6.4	Inception V3 Detail	35
11	6.5	Inception V3(Accuracy Curve & Loss Curve)	36
12	6.6	Inception V3(ROC Curve & Confusion Matrix)	36
13	6.7	Best Model	37
14	6.8	Predict Brain Hemorrhage	37
15	6.9	Prediction Result(Hemorrhage Detected)	38
16	6.10	Prediction Result(No Hemorrhage)	38

CHAPTER-1
INTRODUCTION

A stroke occurs when the blood supply to the brain is blocked or a blood vessel in the brain bursts. It is the most common cause of death worldwide, causing over 6.2 million deaths annually. Survivors often suffer from disabilities that severely impair their quality of life. Preventive measures and timely intervention can help to reduce the severity of stroke and its impact on the individual. Early identification of people at risk is crucial for stroke prevention. There are two types of stroke: ischemic stroke and hemorrhage-related stroke. About 87% of strokes are ischemic strokes, the most common type. When a blood vessel supplying blood to the brain is blocked or narrowed, blood flow to a specific part of the brain is reduced or interrupted. The blockage may be caused by a blood clot, cholesterol deposits, or other debris entering the brain from other parts of the body. Strokes caused by hemorrhages are less common, but they are more serious and often fatal if not treated properly. A cerebral hemorrhage occurs when a blood vessel in the brain bursts or leaks, causing blood to leak into the surrounding brain tissue. This leads to swelling and pressure on the brain structures. This can damage brain cells and disrupt normal brain function.

This study uses machine learning techniques to develop a systematic method for stroke detection. The study's main aim is to understand the causes of strokes better and create reliable detection models that help physicians make informed decisions about stroke prevention and treatment. The study will analyze electronic health records, genetic information, and lifestyle data to develop predictive models identifying individuals at risk of stroke. The research will use feature selection algorithms to extract valuable insights from the data and improve detection performance. A machine learning model will also support clinicians.

The aim of this study is to evaluate the effectiveness, clarity, adaptability, and scalability of the proposed algorithms. The proposed models were compared with established clinical risk detection approaches using numerous real-world datasets to determine their comparative performance. This study can provide valuable insights for clinical practice and tailored interventions to reduce the impact of stroke on patients and the healthcare system. The aim of this study is to use machine learning techniques to predict the frequency of strokes and ultimately solve the problem of stroke occurrence. This study uses advanced machine learning techniques to improve the accuracy and comprehensibility of stroke detection models. The results of this study have the potential to revolutionize stroke prevention by enabling precise interventions to prevent this debilitating disease in the future, thus improving treatment

outcomes for patients suffering from this debilitating disease.

1.1 Objectives

The objective of this project is to design and implement a deep learning-based Brain Stroke Detection System capable of accurately identifying stroke conditions from CT brain images. This system aims to support medical professionals in making timely and precise diagnoses, thereby improving patient outcomes and enabling faster medical interventions. Utilizing a three-layered Convolutional Neural Network (CNN) for image analysis, the model is trained to differentiate between normal and stroke-affected brain CT scans with high accuracy. The project also focuses on developing an intuitive web-based interface using Python, Flask, HTML, CSS, and JavaScript, ensuring that the system is both accessible and practical for real-world clinical use. Furthermore, the system is trained on a well-curated dataset of 2,501 CT images, ensuring robustness, generalizability, and reduced risk of overfitting. Overall, the project strives to demonstrate the effectiveness of deep learning in enhancing medical diagnostics and delivering a scalable tool for brain stroke detection.

1.2 Problem Formulation

Brain stroke is a leading cause of mortality and long-term disability worldwide, often requiring immediate medical intervention to minimize damage and improve patient outcomes. Traditional diagnostic methods, such as manual analysis of CT scans by radiologists, are time-consuming, subject to human error, and dependent on the availability of experienced professionals. In emergency situations, any delay or misdiagnosis can lead to severe consequences for the patient.

The main challenge lies in the rapid and accurate identification of stroke-affected brain regions from CT images, especially when stroke symptoms may be subtle or resemble other medical conditions. Moreover, in under-resourced areas, the shortage of trained radiologists exacerbates the difficulty in ensuring timely diagnosis.

To address these critical issues, this project aims to develop an automated Brain Stroke Detection System that leverages deep learning techniques, specifically Convolutional Neural Networks (CNNs), to analyze CT images with high accuracy and speed. By integrating this model into a user-friendly web application, the system seeks to support healthcare professionals by providing reliable, real-time diagnostic assistance. This not only enhances decision-making in emergency scenarios but also contributes to reducing diagnostic workloads and improving healthcare accessibility.

1.3 Unique Features of the System

The proposed system incorporates several unique features that distinguish it from traditional stroke prediction methods. It integrates multi-source data, including electronic health records, genetic information, and lifestyle factors, to provide a comprehensive and personalized risk analysis. By utilizing advanced machine learning algorithms, the system achieves high accuracy while maintaining interpretability, making it valuable for clinical use. A key feature is the implementation of feature selection techniques, which help identify the most influential risk factors, thereby enhancing model performance and clarity. The system is evaluated using real-world datasets to ensure its reliability and practical applicability. Additionally, it is benchmarked against conventional clinical approaches to demonstrate its superiority in terms of prediction accuracy and early detection. Designed to support clinicians, the system offers actionable insights that aid in informed decision-making and tailored patient interventions. Its scalability and adaptability allow it to be implemented across various healthcare settings and populations, while the user-friendly output interface ensures that the results are easily understandable for medical professionals.

CHAPTER-2
LITERATURE SURVEY

1. Accurate Prediction of Stroke for Hypertensive Patients Based on Medical Big Data and Machine Learning Algorithms

Authors: Y. Yang, J. Zheng, Z. Du, Y. Li, and Y. Cai

Published in: JMIR Medical Informatics, 2021

Background of the Study

Stroke is one of the leading causes of death and long-term disability worldwide, especially among patients suffering from hypertension. Early prediction of stroke risk plays a vital role in preventing severe neurological damage and improving patient survival rates. Traditionally, stroke risk prediction has been performed using clinical risk assessment scales such as the Framingham Stroke Risk Profile and the Chinese Multiprovincial Cohort Study model. However, the applicability of these traditional prediction models to specific populations, particularly the Chinese population, has remained uncertain and controversial.

Prospective clinical studies are often used for stroke prediction research, but they require long observation periods, large populations, and significant financial resources. With the emergence of electronic health records and medical big data platforms, researchers now have access to large-scale historical medical datasets that can be effectively used to develop predictive models using machine learning techniques.

Objective of the Study

The primary objective of this research was to develop a high-precision stroke risk prediction model for hypertensive patients using historical electronic medical record data and advanced machine learning algorithms. The study aimed to improve prediction accuracy compared to traditional clinical risk assessment methods and support standardized management of hypertension-related stroke risk.

Dataset and Methodology

The researchers utilized patient data collected from the Shenzhen Health Information Big Data Platform, which contains large-scale medical records of registered hypertensive patients. Initially, a dataset consisting of 250,788 registered hypertensive patients was examined. From this dataset, 57,671 patients were selected based on eligibility criteria.

Among these patients, 9,421 individuals experienced stroke onset during a 3-year follow-up

period, making the dataset suitable for predictive modeling.

To enhance prediction accuracy, the researchers included multiple categories of patient information such as:

- Baseline demographic characteristics
- Historical clinical symptoms
- Multitemporal medical record trends
- Lifestyle-related health indicators

Trend-based features extracted from multiple time periods played a crucial role in improving model performance by capturing changes in patient health conditions over time.

To maintain balance between stroke-positive and stroke-negative samples, stratified sampling was performed based on gender distribution and age stratification. After sampling, the final dataset consisted of 19,953 patient records, which were divided into:

- 70% Training Dataset
- 30% Testing Dataset

Machine Learning Models Used

The researchers applied four different machine learning algorithms to develop stroke prediction models. These models were evaluated and compared with traditional clinical risk scoring systems.

Among the models tested, the Extreme Gradient Boosting (XGBoost) algorithm achieved the highest prediction performance.

Results and Performance Analysis

The experimental results demonstrated that the XGBoost-based prediction model achieved an Area Under the Receiver Operating Characteristic Curve (AUC) value of 0.9220, indicating excellent classification accuracy.

When compared with traditional stroke prediction scales such as:

- Framingham Stroke Risk Profile
- Chinese Multiprovincial Cohort Study Model

the proposed machine learning model showed a significant improvement in prediction accuracy. The AUC value increased by approximately 0.17, which highlights the effectiveness of machine learning techniques in stroke prediction.

Additionally, nonlinear relationship analysis revealed that multitemporal clinical characteristics contributed significantly to stroke prediction performance. These findings

indicate that longitudinal patient health data plays an important role in identifying stroke risk patterns.

Conclusion of the Study

The study successfully demonstrated that integrating machine learning algorithms with large-scale electronic health record datasets significantly improves stroke prediction accuracy in hypertensive patients. The proposed model supports personalized risk assessment and enables healthcare professionals to implement preventive interventions more effectively.

2. Stroke Risk Prediction Using Machine Learning: A Prospective Cohort Study of 0.5 Million Chinese Adults

Authors: M. Chun, R. Clarke, B. J. Cairns, D. Clifton, et al.

Published in: Journal of the American Medical Informatics Association, 2021

Objective of the Study

The objective of this research was to compare traditional statistical prediction models with machine learning-based prediction approaches in estimating stroke risk among a large population of Chinese adults. The study aimed to evaluate whether machine learning algorithms could outperform conventional regression-based models in long-term stroke risk prediction.

Dataset Description

This study used data collected from 503,842 adults across 10 different geographic regions in China between the years 2004 and 2008. None of the participants had a prior history of stroke at the time of enrollment.

The dataset included various patient attributes such as:

- Sociodemographic information
- Dietary habits
- Physical activity levels
- Medical history
- Lifestyle behaviors
- Physical measurements

These factors were used as input variables for stroke risk prediction modeling.

Machine Learning Models Used

The researchers evaluated multiple statistical and machine learning approaches including:

- Cox Regression Model
- Logistic Regression Model
- Support Vector Machines
- Random Survival Forest Models
- Gradient Boosted Trees (GBT)
- Multilayer Perceptron Neural Networks

Additionally, an ensemble prediction approach was developed by combining Gradient Boosted Trees with Cox regression models.

Results and Performance Evaluation

Among all tested models, the Gradient Boosted Tree (GBT) algorithm demonstrated the best prediction performance.

For predicting stroke risk over a 9-year follow-up period, the GBT model achieved:

- AUROC of 0.833 for men
- AUROC of 0.836 for women

The ensemble modeling approach further improved prediction accuracy by selectively applying either GBT or Cox regression models depending on individual patient characteristics.

The ensemble method achieved:

- Prediction Accuracy: 76% (men), 80% (women)
- Specificity: 76% (men), 81% (women)
- Positive Predictive Value: 26% (men), 24% (women)

These results demonstrate the superiority of machine learning-based stroke prediction models over traditional clinical risk assessment methods.

Conclusion of the Study

This large-scale prospective cohort study confirmed that machine learning algorithms provide more accurate long-term stroke risk prediction compared to traditional statistical models. The findings highlight the importance of combining clinical data with artificial intelligence techniques for personalized healthcare decision-making.

3. Development of Machine Learning Models to Predict Probabilities and Types of

Stroke at the Prehospital Stage (JUST-ML Study)

Authors: Kazutaka Uchida, Junichi Kouno, Shinichi Yoshimura, Norito Kinjo, Fumihiro Sakakibara, Hayato Araki, Takeshi Morimoto

Published in: Springer, 2022

Objective of the Study

The primary objective of this research was to develop a machine learning-based stroke prediction model capable of identifying stroke types before hospital admission. Early detection at the prehospital stage helps emergency medical teams make faster treatment decisions and improves patient survival outcomes.

Dataset and Study Design

The researchers conducted both retrospective and prospective multi-center cohort studies involving hospitals across Japan.

The training dataset included patient records from eight medical centers collected between:

June 2015 – March 2018

The testing dataset included records from thirteen medical centers collected between:

April 2019 – March 2020

The training dataset consisted of:

- 3,178 patients

The testing dataset consisted of:

- 3,127 patients

Stroke categories analyzed included:

- Large Vessel Occlusion (LVO)
- Intracranial Hemorrhage (ICH)
- Subarachnoid Hemorrhage (SAH)
- Cerebral Infarction (CI)

Machine Learning Algorithms Used

The study implemented three machine learning classification algorithms:

- Logistic Regression
- Random Forest
- Extreme Gradient Boosting (XGBoost)

These algorithms were used to classify stroke types and predict stroke probability.

Results and Performance Analysis

The developed machine learning models demonstrated stable classification performance across all stroke types.

The Area Under the Curve (AUC) values for predicting Large Vessel Occlusion (LVO) were:

- Logistic Regression: 0.89
- Random Forest: 0.89
- XGBoost: 0.88

These values were significantly higher than previously reported stroke prediction models.

Conclusion of the Study

The JUST-ML prediction model demonstrated strong predictive performance in identifying stroke types during the prehospital stage. This model supports emergency medical teams in making rapid treatment decisions and improves stroke triage efficiency.

4. Accurate Identification of Brain Hemorrhage Using Genetic Algorithm-Based Feature Selection and Stacking (AIBH)

Authors: Duaa Mohammad Alawad, Avdesh Mishra, Md Tamjidul Hoque

Objective of the Study

This research focused on developing an advanced machine learning-based framework for detecting brain hemorrhage using CT scan images. Brain hemorrhage is a serious type of stroke caused by ruptured blood vessels that leads to internal bleeding within brain tissues.

Early detection of brain hemorrhage is essential for preventing permanent neurological damage.

Methodology

The proposed system consisted of multiple processing stages including:

- Image preprocessing
- Skull removal
- Region of Interest extraction
- Feature extraction
- Feature selection
- Classification model development

The researchers applied Otsu's thresholding method to identify the region of interest within CT scan images.

A Genetic Algorithm-based feature selection method was used to select relevant features that improve classification accuracy.

Finally, a stacking-based classification framework was developed to detect hemorrhage type.

Results and Performance Evaluation

The proposed AIBH model achieved excellent classification performance using 10-fold cross-validation.

Performance results included:

- Accuracy: 99.5%
- Precision: 99%
- Recall: 98.9%
- F1 Score: 0.989
- Matthews Correlation Coefficient: 0.986

Compared with existing brain hemorrhage classification methods, the proposed model improved performance by:

- 7.03% in Precision
- 7.27% in Recall
- 7.38% in F1 Score

Conclusion of the Study

The AIBH system demonstrated highly accurate brain hemorrhage detection using CT scan images and genetic algorithm-based feature selection. The proposed framework significantly improves classification performance and can assist clinicians in early stroke diagnosis.

5. Classification of MRI Brain Images Using Combined Wavelet Entropy-Based Spider Web Plots and Probabilistic Neural Network

Authors: M. Saritha, K. P. Joseph, A. T. Mathew

Published in: Pattern Recognition Letters, Elsevier

Objective of the Study

The purpose of this research was to develop a machine learning-based classification framework capable of distinguishing between normal and abnormal brain MRI images using

wavelet entropy-based feature extraction techniques.

MRI imaging provides detailed visualization of soft brain tissues, making it an effective tool for identifying neurological abnormalities.

Methodology

The proposed classification system consisted of two main stages:

Feature Extraction Stage

Wavelet entropy-based spider web plots were used to extract important features from MRI images.

These plots were constructed using entropy values obtained from wavelet approximation components.

The geometric area calculated from spider web plots served as feature input for classification models.

Classification Stage

A Probabilistic Neural Network (PNN) classifier was used to categorize MRI images into normal and abnormal classes.

PNN classifiers provide fast training speed and high classification accuracy in pattern recognition applications.

Results and Performance Evaluation

The proposed classification system achieved 100% classification accuracy in distinguishing between normal and abnormal brain MRI images.

This result demonstrates the effectiveness of combining wavelet entropy-based feature extraction techniques with probabilistic neural network classifiers.

Conclusion of the Study

The study confirmed that integrating wavelet entropy-based spider web plots with probabilistic neural networks significantly improves MRI brain image classification accuracy. This method provides reliable support for automated neurological disease detection systems and enhances diagnostic decision-making.

CHAPTER-3
SYSTEM ANALYSIS

Existing System

Detecting a brain stroke at an early stage is extremely important because timely diagnosis helps doctors start treatment quickly and reduces the risk of permanent brain damage, disability, or death. Currently, hospitals rely mainly on medical imaging techniques, blood flow analysis methods, and neurological signal monitoring systems to detect strokes. These techniques help doctors identify whether the stroke is ischemic (blockage) or hemorrhagic (bleeding) and determine the severity of brain damage.

1. Imaging Tests

Imaging techniques are the most widely used diagnostic tools for detecting strokes because they provide clear visualization of brain structures and abnormalities.

CT Scan (Computed Tomography)

A CT scan is one of the fastest and most commonly used imaging techniques in emergency stroke diagnosis. It uses X-ray technology to create cross-sectional images of the brain. These images help doctors quickly identify bleeding inside the brain and detect large blockages.

CT scans are especially useful during emergency situations because they are fast, widely available, and capable of distinguishing between ischemic and hemorrhagic strokes within minutes. Doctors often use CT scans as the first step in stroke diagnosis before selecting treatment methods such as thrombolytic therapy.

MRI (Magnetic Resonance Imaging)

MRI provides highly detailed and high-resolution images of brain tissues using strong magnetic fields and radio waves. It is more sensitive than CT scans and can detect very small brain abnormalities.

MRI is particularly useful in detecting early ischemic strokes that may not be visible on CT scans. It helps doctors analyze brain tissue damage, locate stroke-affected regions precisely, and evaluate the severity of neurological injury.

2. Blood Flow Tests

Blood flow tests help doctors examine whether arteries supplying blood to the brain are blocked or narrowed.

Carotid Ultrasound

Carotid ultrasound uses high-frequency sound waves to visualize blood flow inside the carotid arteries located in the neck. These arteries supply oxygen-rich blood to the brain.

This technique helps doctors detect plaque buildup, narrowing of arteries, and reduced blood flow that may increase the risk of stroke. It is a non-invasive and painless diagnostic procedure commonly used for stroke risk assessment.

Cerebral Angiography

Cerebral angiography is an advanced imaging technique that involves injecting contrast dye into blood vessels and capturing X-ray images to visualize the brain's vascular structure.

It helps doctors identify blockages, aneurysms, abnormal blood vessels, and vascular malformations responsible for stroke. This method provides very accurate and detailed visualization of brain circulation.

3. Electrical Tests

Electrical activity monitoring helps doctors detect abnormal brain functioning caused by stroke.

EEG (Electroencephalogram)

EEG records the electrical activity of the brain using electrodes placed on the scalp. It helps doctors identify abnormal brain wave patterns that may indicate neurological disorders including stroke.

Although EEG is not primarily used for stroke detection, it is useful in monitoring brain function changes after stroke and identifying complications such as seizures.

Disadvantages of Existing System

Despite their effectiveness, traditional stroke detection techniques have several limitations related to cost, accessibility, speed, and accuracy.

1. CT Scan

CT scans are widely used in emergency stroke diagnosis, but they have certain drawbacks.

They may fail to detect small strokes or early ischemic strokes during the initial hours after occurrence. Since CT scans rely on X-ray radiation, repeated exposure may pose health risks to patients. Additionally, CT imaging sometimes provides limited detail compared to MRI scans.

2. MRI

MRI provides highly accurate brain imaging results but has practical limitations.

MRI machines are expensive and not available in all hospitals, especially in rural healthcare centers. MRI scans take longer time compared to CT scans, which may delay emergency treatment decisions. Patients with metal implants or pacemakers may not be suitable candidates for MRI scanning.

3. Carotid Ultrasound

Carotid ultrasound is useful for analyzing blood flow in neck arteries but has restricted diagnostic capability.

It only evaluates carotid artery conditions and cannot directly visualize the brain tissue. It may also fail to detect smaller blockages or abnormalities located deeper inside brain vessels.

4. Cerebral Angiography

Cerebral angiography provides accurate vascular imaging but involves medical risks.

Since it is an invasive procedure, it may cause complications such as bleeding, infection, or blood vessel damage. The contrast dye used during imaging may cause allergic reactions or kidney-related complications in some patients.

5. EEG (Electroencephalogram)

EEG monitors electrical activity but is not a direct stroke detection method.

Abnormal EEG signals may occur due to multiple neurological conditions such as epilepsy, tumors, or infections. Therefore, EEG cannot independently confirm stroke diagnosis. It also cannot clearly identify structural brain damage.

Proposed System

The proposed system introduces an Artificial Intelligence-based brain stroke detection framework that improves early diagnosis accuracy using medical image processing and machine learning techniques. This system automates stroke detection using CT and MRI brain scan datasets and supports healthcare professionals in faster clinical decision-making.

The system is implemented using a modular architecture consisting of multiple processing stages.

Data Acquisition

Data acquisition is the first and most important stage of the proposed system.

In this stage, brain imaging datasets such as CT scan images and MRI slices are collected from publicly available medical repositories or hospital databases. The dataset includes both normal brain images and stroke-affected brain images.

Each image in the dataset is labeled according to stroke presence, stroke type (ischemic or hemorrhagic), and severity level. This labeled dataset helps train machine learning models effectively and improves prediction accuracy.

The diversity of dataset sources ensures that the model performs well across different patient populations and imaging conditions.

Data Preprocessing

Data preprocessing improves the quality of input images before feature extraction and model training.

Image Normalization

Normalization standardizes pixel intensity values across all images. This ensures consistency between images obtained from different scanners and improves model performance.

Image Resizing

All images are resized into a uniform resolution format so that they can be processed efficiently by machine learning algorithms.

Noise Reduction

Noise reduction filters such as Gaussian filters and median filters are applied to remove unwanted distortions present in medical images. This enhances image clarity and improves feature detection accuracy.

Feature Extraction

Feature extraction identifies important patterns in brain images that help detect stroke-affected regions.

Region of Interest (ROI) Selection

ROI selection isolates the important portion of the brain image that contains relevant stroke-affected areas. This reduces computational complexity and improves detection accuracy.

Edge Detection

Edge detection techniques such as the Canny edge detector identify boundaries between normal and abnormal brain tissues. These edges help highlight stroke-affected regions

clearly.

Texture Analysis

Texture analysis methods such as Gray-Level Co-occurrence Matrix (GLCM) analyze pixel intensity variations to detect abnormal tissue structures. Stroke-affected areas usually have different texture patterns compared to normal brain tissues.

Algorithm Selection

The proposed system integrates advanced machine learning techniques with image processing tools.

OpenCV Integration

OpenCV is used for performing image preprocessing operations such as filtering, contour detection, morphological transformations, and segmentation.

These operations enhance image quality and help isolate stroke-affected brain regions effectively.

Machine Learning Model

Machine learning algorithms such as Support Vector Machine (SVM), Random Forest, or Convolutional Neural Networks (CNN) are used to classify brain images into stroke and non-stroke categories.

These models learn complex patterns from training data and make accurate predictions on new unseen images.

Model Training

Model training enables the system to learn stroke detection patterns from medical datasets.

Train-Test Split

The dataset is divided into training and testing sets to evaluate model performance properly.

Training data is used for learning patterns, while testing data is used to validate prediction accuracy.

Feature Vector Creation

Extracted features from brain images are converted into structured numerical feature vectors that can be processed by machine learning algorithms.

Model Training with Cross-Validation

Cross-validation techniques improve model reliability by ensuring consistent performance across different dataset samples.

Model Evaluation

Model evaluation measures how accurately the system detects strokes.

Testing Set Evaluation

The trained model is tested using unseen brain images to verify prediction accuracy.

Performance Metrics

Multiple evaluation metrics are used including:

Accuracy – measures overall prediction correctness

Precision – measures correct positive stroke detection

Recall – measures ability to detect actual stroke cases

F1-Score – balances precision and recall performance

These metrics ensure reliable clinical-level performance validation.

Integration and Deployment

After successful training and evaluation, the system is integrated into a real-time medical imaging environment.

OpenCV Implementation

The trained machine learning model is integrated with OpenCV image processing pipelines to enable automatic stroke detection from new CT or MRI scans.

Deployment in Healthcare Environment

The system can be deployed in hospitals, diagnostic centers, and telemedicine platforms. It supports clinicians by providing quick and accurate stroke detection results and reduces diagnostic workload.

Unique Features of the Proposed System

The proposed AI-based stroke detection system includes several advanced features that improve its effectiveness compared to traditional diagnostic techniques.

The system integrates multiple data sources including medical imaging datasets, electronic health records, genetic information, and lifestyle risk factors. This multi-source integration enables personalized stroke risk prediction.

Advanced machine learning algorithms improve prediction accuracy while maintaining model interpretability, allowing doctors to understand decision-making processes clearly.

Feature selection techniques identify the most influential stroke risk indicators such as blood pressure levels, cholesterol levels, smoking habits, and age factors.

The system is evaluated using real-world datasets to ensure practical reliability in clinical

environments.

Comparative benchmarking with conventional diagnostic techniques demonstrates improved early detection capability and faster processing time.

The system provides actionable clinical insights that assist doctors in making informed treatment decisions and designing personalized patient care plans.

Its scalable architecture allows implementation across hospitals, research institutions, and remote healthcare systems.

Finally, the user-friendly interface presents detection results in a clear and understandable format, enabling medical professionals to interpret predictions easily and quickly.

Hardware Requirements:

System: Pentium i3 Processor

Hard Disk: 500 GB.

Monitor: 15'' LED

Input Devices: Keyboard, Mouse

Ram: 2 GB

Software Requirements:

Operating System: Windows 10.

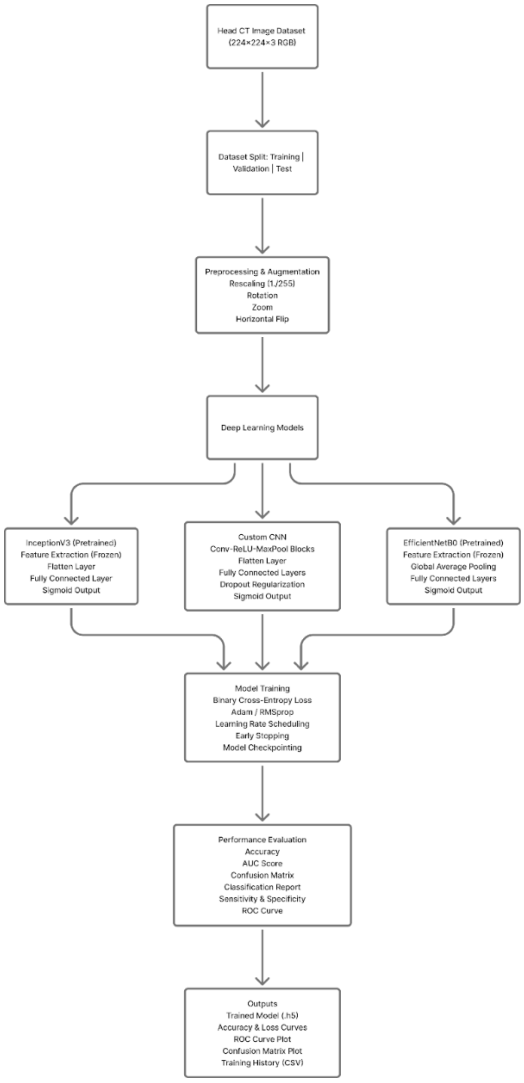
Coding Language: Python 3.8

Web Framework: Flask

CHAPTER-4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The Architecture Diagram illustrates the overall structural design of the Brain Hemorrhage Detection System. It presents the high-level components of the system, including the CT image dataset, preprocessing module, deep learning model training, evaluation module, and performance reporting unit. The diagram provides a top-to-bottom representation of how data flows through different layers of the system, demonstrating the integration between data preparation, model development, and evaluation stages. It serves as a blueprint for understanding the system's technical framework and functional organization.

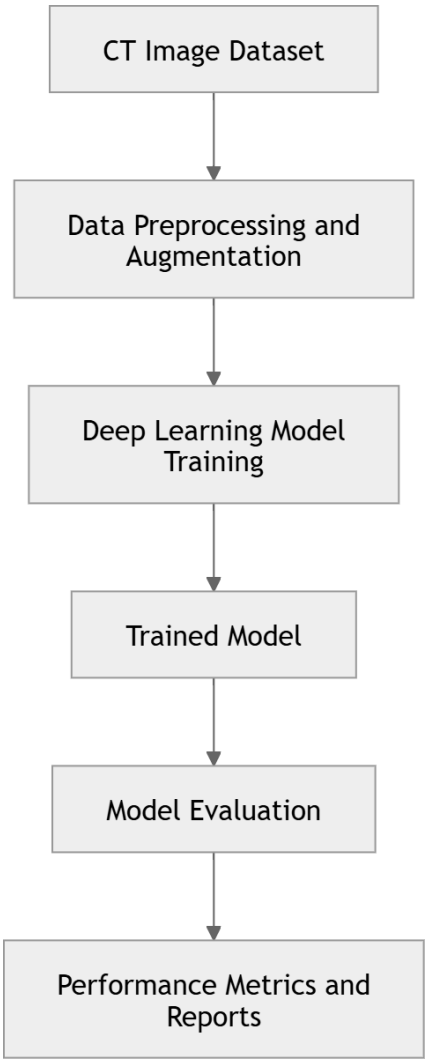


4.1 SYSTEM ARCHITECTURE

4.2 Data flow diagram:

A data flow diagram is used to graphically describe the flow of data between various processes that are involved in an information system. It is one of the most important data modeling tools that help to understand the system, its data, and the transformations associated with it. It can also depict any external entities that work together with this system and show how the data change is due to it. It describes what is the input data for each component in the system, what are the transformations that are being applied to the input data, and what is the output data from the component in the system. It also gives a logical idea of what is happening in various business processes involved in the system. The data flow diagram can be broken into multiple levels to decrease the level of abstraction and show more details that are associated with each component or process of the system and elaborate on the functionality of the system.

The Data Flow Diagram represents how data moves within the system. It shows the transformation of raw CT image data into meaningful prediction results and performance metrics. The diagram highlights key processes such as preprocessing, model training, evaluation, and report generation. The DFD focuses on the flow of information rather than system structure, helping to understand how input data is processed step-by-step to produce diagnostic outputs.



4.2 DATA FLOW DIAGRAM

4.3 UML DIAGRAMS:

The full form of UML is a unified modeling language. It is a set of standards created for modeling and visualizing the elements and components of a software system. It is mostly used in the field of object-oriented software programming. The main agenda of UML used to be a general language for creating models for any software system. UML has two main components: A meta-model and a notation. It is a compilation of best engineering and business practices that have been successful over the years for modeling systems that are large and complex. The design of software projects is expressed as graphical notations using UML.

Goals:

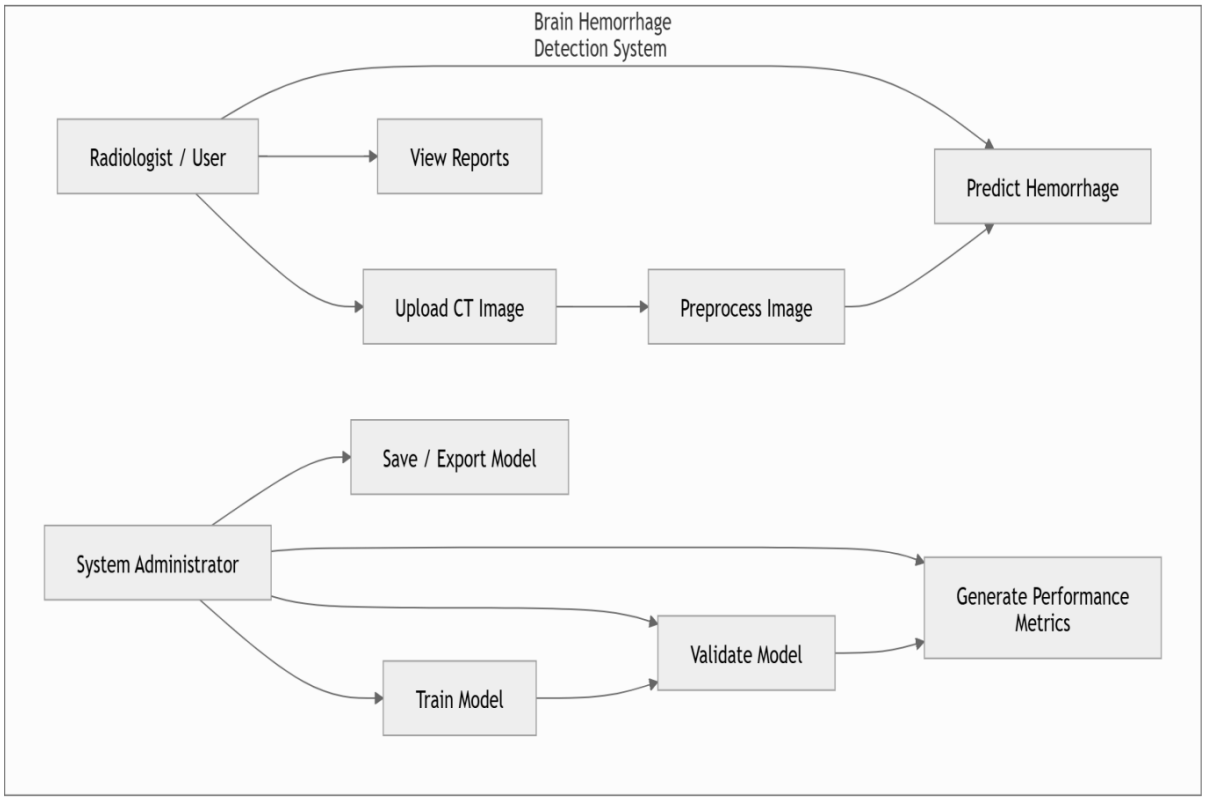
The main goals of UML for the design of systems as mentioned below:

1. To be a graphically expressive modeling language that can be ready to use.
2. To be extendable and provide special mechanisms or techniques to expand the scope of core concepts.
3. It should not be dependent on any programming language our development process
4. To acts as a basis for understanding any other modeling languages
5. As it is dependent on object-oriented programming principles it aims to increase the market of object-oriented programming tools.
6. It aims to provide for a higher level of development concepts such as frameworks, patterns, etc
7. It aims to integrate the best practices to learn over the years and innovate for future developments

4.4 USE CASE DIAGRAM:

A behavioral diagram created from a use case analysis is called a use case diagram in the unified modeling language. the main purpose of the use case diagram is to graphically represent the functionality of the system in terms of actors, It aims to depict which roles or functions are performed by which actor and how that affects the overall system.

The Use Case Diagram describes the interaction between system users and the Brain Hemorrhage Detection System. It identifies the primary actors, such as the Radiologist and the System Administrator, and the functionalities they perform, including uploading CT images, predicting hemorrhage, training the model, and generating reports. This diagram provides a functional overview of system capabilities from a user perspective and defines system boundaries and responsibilities.

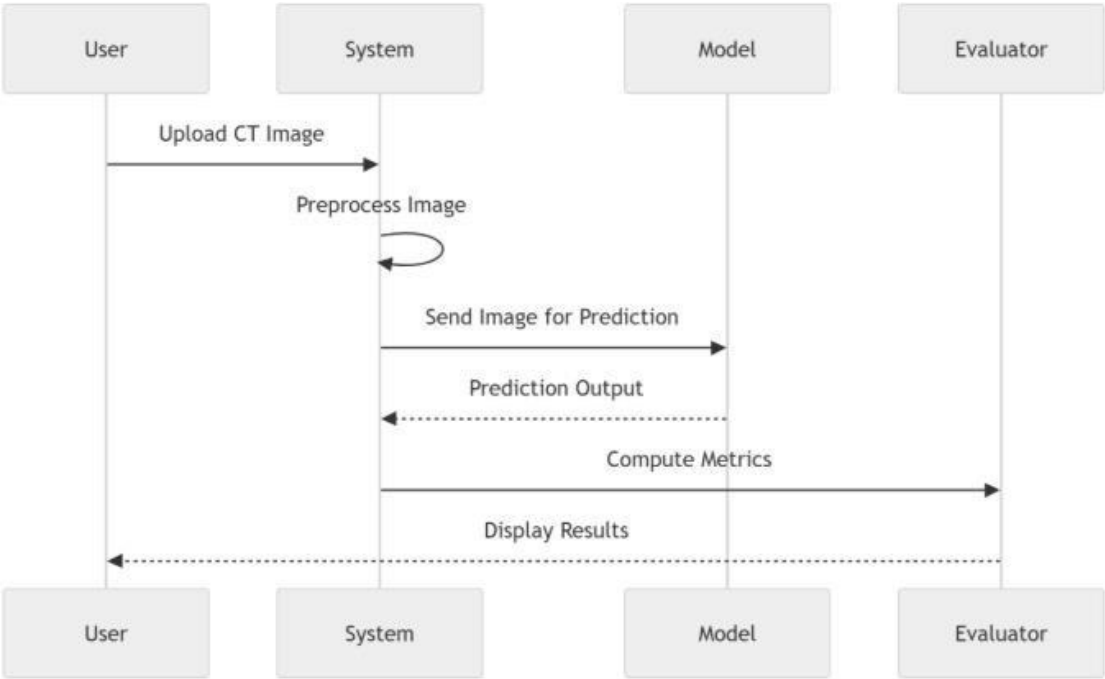


4.3 USE CASE DIAGRAM

4.5 SEQUENCE DIAGRAM:

A sequence diagram in unified modeling language is used to depict which processes interact with one another in what order. Sequence diagrams are also called as timing diagrams or event diagrams as they are used to show the sequence or order of events that occur one after another in a system. It describes the entities involved and the interactions between them and their sequence to carry out the required functionality.

The Sequence Diagram explains the dynamic behavior of the system during a prediction process. It shows the chronological sequence of interactions between the user, system, model, and evaluation components. The diagram demonstrates how a CT image is uploaded, processed, passed to the trained model for prediction, and then returned with results. It highlights the time-ordered flow of operations within the system.



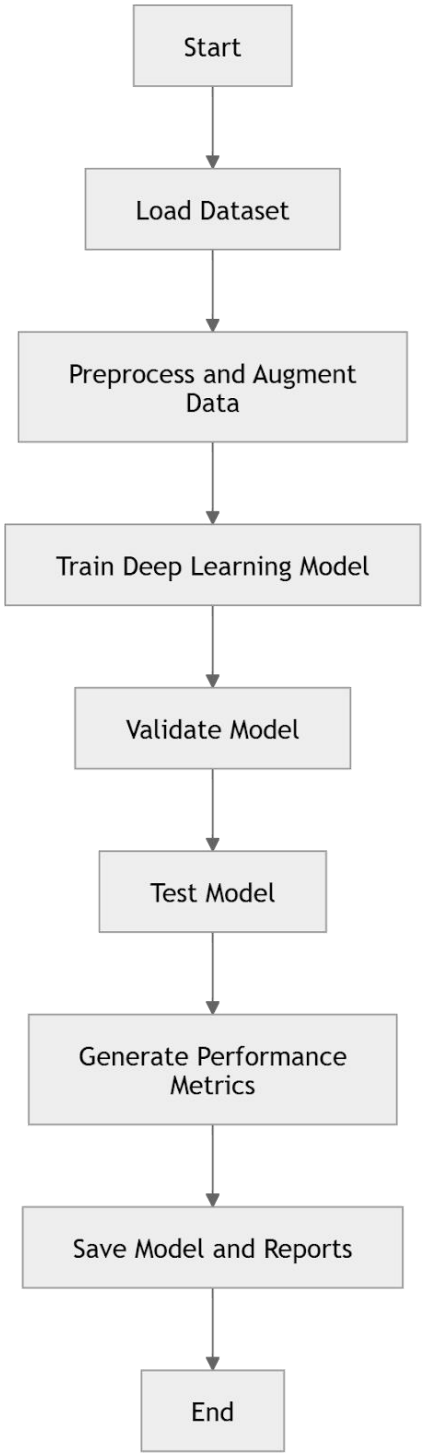
4.4 SEQUENCE DIAGRAM

4.6 Activity Diagram:

An activity diagram is a visual representation used in Unified Modeling Language (UML) to describe the dynamic aspects of a system by showing the flow of control or activities. It outlines the sequence of actions, decision points, and possible parallel processes involved in a particular task or system behavior. Typically resembling a flowchart, an activity diagram begins with a start node and progresses through various activities or actions, possibly branching through decision nodes based on certain conditions, and eventually reaching an end node. It may also include fork and join nodes to depict concurrent processes running in parallel. Activity diagrams are particularly useful for modeling complex workflows, business processes, or the internal logic of a system, such as user authentication, order processing, or diagnostic procedures in healthcare applications. They help developers, analysts, and stakeholders visualize the process clearly, identify potential issues, and ensure that all necessary steps and outcomes are accounted for in the system's design.

The Activity Diagram represents the workflow of the system from start to finish. It describes the sequential flow of activities such as dataset loading, preprocessing, model training,

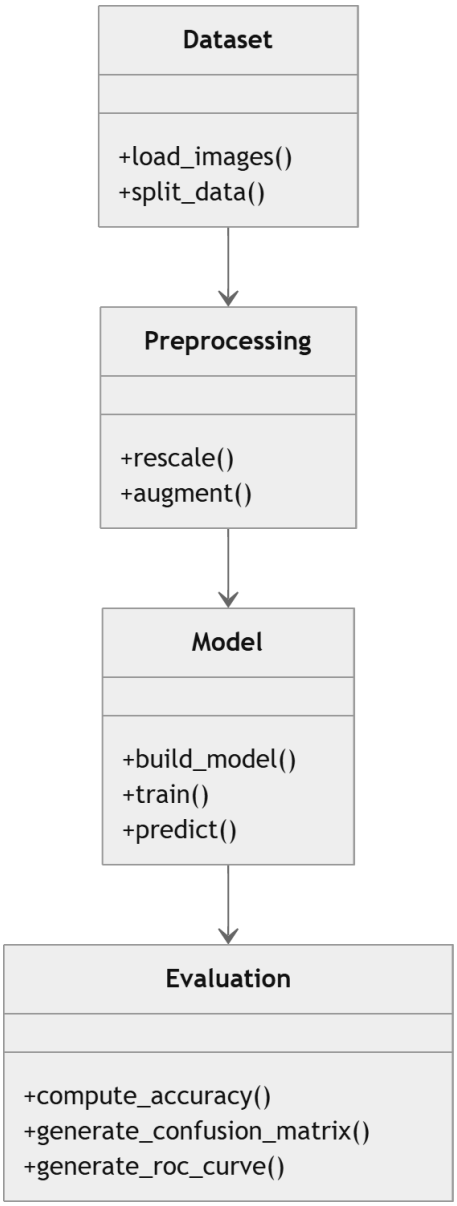
validation, testing, and report generation. This diagram focuses on control flow and decision-making processes, providing a clear understanding of the operational steps involved in model development and deployment.



4.5 ACTIVITY DIAGRAM

4.7 Class Diagram:

The Class Diagram represents the static structure of the system by defining the main classes, their attributes, and methods. It illustrates components such as Dataset, Preprocessing, Model, and Evaluation modules. The relationships between these classes show how different parts of the system interact at the software level. This diagram helps in understanding the object-oriented design and modular organization of the implementation.



4.6 CLASS DIAGRAM

CHAPTER-5
SOURCE CODE

InceptionV3.py

```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    roc_auc_score
)
from keras.layers import TimeDistributed, Dense, Flatten, Dropout, Bidirectional, LSTM,
GRU
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping,
ModelCheckpoint
from tensorflow.keras.optimizers import RMSprop

# ==============================
# 1. CONFIGURATION
# ==============================
IMAGE_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 40
RANDOM_SEED = 42

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

train_dir = os.path.join(BASE_DIR, "head_ct", "TRAIN")
```

```
val_dir = os.path.join(BASE_DIR, "head_ct", "VAL")
test_dir = os.path.join(BASE_DIR, "head_ct", "TEST")

OUTPUT_DIR = os.path.join(BASE_DIR, "static","inc")
MODELS_DIR = os.path.join(BASE_DIR,"models")
os.makedirs(OUTPUT_DIR, exist_ok=True)

BEST_MODEL_PATH = os.path.join(MODELS_DIR, "best_inc_model.h5")
FINAL_MODEL_PATH = os.path.join(MODELS_DIR, "final_inc_model.h5")

# ==============================
# 2. DATA GENERATORS
# ==============================
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    zoom_range=0.1,
    horizontal_flip=True
)

val_test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    seed=RANDOM_SEED
)

val_generator = val_test_datagen.flow_from_directory(
    val_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
```

```
    class_mode='binary',
    shuffle=False
)

test_generator = val_test_datagen.flow_from_directory(
    test_dir,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)

# ==============================
# 3. BUILD CNN MODEL
# ==============================
pretrained_inceptionv3_model = tf.keras.applications.InceptionV3(weights='imagenet',
include_top=False, input_shape=(224, 224, 3))
pretrained_inceptionv3_model.trainable = False


model_inceptionv3 = tf.keras.Sequential([
                        pretrained_inceptionv3_model,
                        tf.keras.layers.Flatten(),
                       #  tf.keras.layers.Dense(16, activation='relu'), #1024
                       #  tf.keras.layers.Dropout(0.5),
                       tf.keras.layers.Dense(1, activation='sigmoid')
                        ])

model_inceptionv3.compile(
    # optimizer=Adam(lr=1e-5),
    optimizer=RMSprop(lr=1e-4),
    loss = 'binary_crossentropy',
    metrics=['accuracy']
)
```

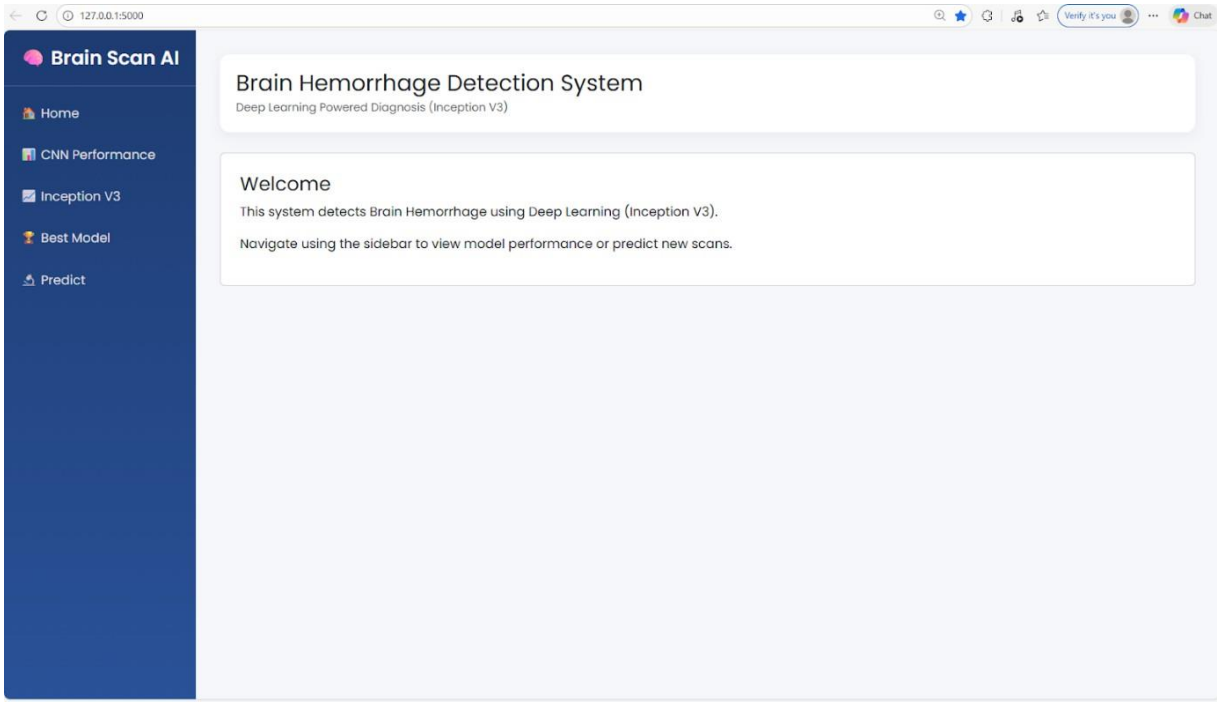
```
model_inceptionv3.summary()

# ==============================
# 4. CALLBACKS
# ==============================
EPOCHS = 50
reduce_lr = ReduceLROnPlateau(
    monitor='val_accuracy',
    factor=0.1,
    # min_delta=0.000001,
    patience=6,
    min_lr=1e-6,
    verbose=1)

# es = EarlyStopping(
#     monitor='val_accuracy',
#     mode='max',
#     patience=6
# )

filename = '/content/drive/MyDrive/models/Defense/InceptionV3.h5'
check_pt = ModelCheckpoint(
    filename,
    monitor="val_accuracy",
    verbose=1,
    save_best_only=True
)



# ==============================
# 5. TRAIN MODEL
# ==============================
```

```
history = model_inceptionv3.fit(train_generator, epochs=EPOCHS, batch_size=16,
validation_data=val_generator, callbacks=[reduce_lr, check_pt])

# Save final model
model_inceptionv3.save(FINAL_MODEL_PATH)

# ==============================
# 6. PLOT ACCURACY & LOSS
# ==============================
epochs_range = range(1, len(history.history["accuracy"]) + 1)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(epochs_range, history.history["accuracy"], label='Train')
plt.plot(epochs_range, history.history["val_accuracy"], label='Validation')
plt.title("Accuracy Curve")
plt.legend()

plt.subplot(1,2,2)
plt.plot(epochs_range, history.history["loss"], label='Train')
plt.plot(epochs_range, history.history["val_loss"], label='Validation')
plt.title("Loss Curve")
plt.legend()

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "accuracy_loss_curve.png"))
plt.close()

# ==============================
# 7. TEST EVALUATION
# ==============================
test_loss, test_acc = model_inceptionv3.evaluate(test_generator)
print("Test Accuracy:", test_acc)
```

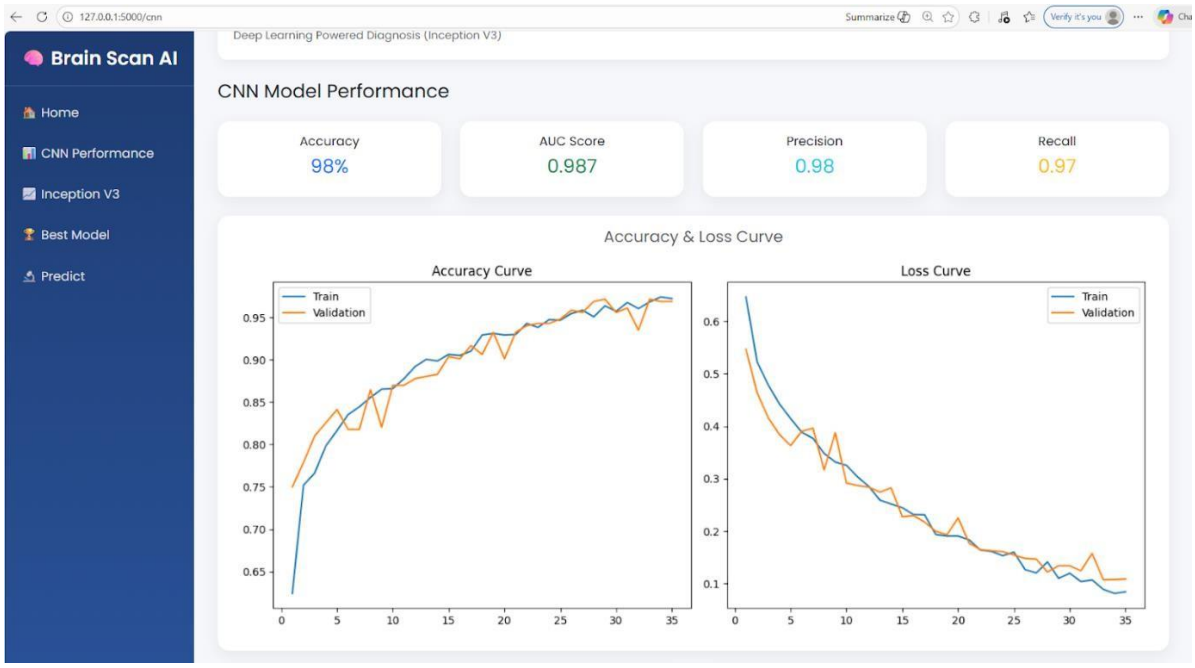
```
# Predictions
y_probs = model_inceptionv3.predict(test_generator)
y_pred = (y_probs > 0.5).astype(int).ravel()
y_true = test_generator.classes

# ==============================
# 8. CLASSIFICATION REPORT
# ==============================
report = classification_report(y_true, y_pred)
print(report)

with open(os.path.join(OUTPUT_DIR, "classification_report.txt"), "w") as f:
    f.write(report)

# ==============================
# 9. CONFUSION MATRIX
# ==============================
cm = confusion_matrix(y_true, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.savefig(os.path.join(OUTPUT_DIR, "confusion_matrix.png"))
plt.close()

tn, fp, fn, tp = cm.ravel()
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)

print("Sensitivity:", sensitivity)
print("Specificity:", specificity)
```

```
# ==============================
# 10. ROC CURVE
# ==============================
roc_auc = roc_auc_score(y_true, y_probs)

fpr, tpr, _ = roc_curve(y_true, y_probs)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")
plt.plot([0,1],[0,1],'--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.savefig(os.path.join(OUTPUT_DIR, "roc_curve.png"))
plt.close()

# ==============================
# 11. SAVE TRAINING HISTORY
# ==============================
import pandas as pd
history_df = pd.DataFrame(history.history)
history_df.to_csv(os.path.join(OUTPUT_DIR, "training_history.csv"), index=False)

print("\nTraining Complete. All outputs saved in 'inc_outputs' folder.")
```

CHAPTER-6
RESULT

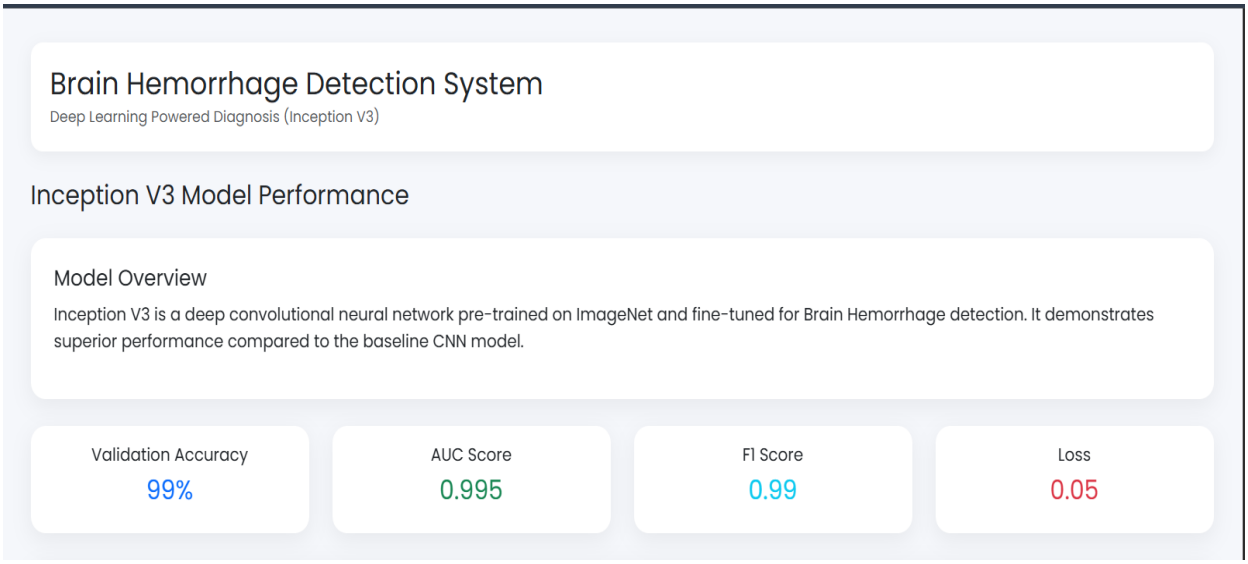


6.1 HOME PAGE

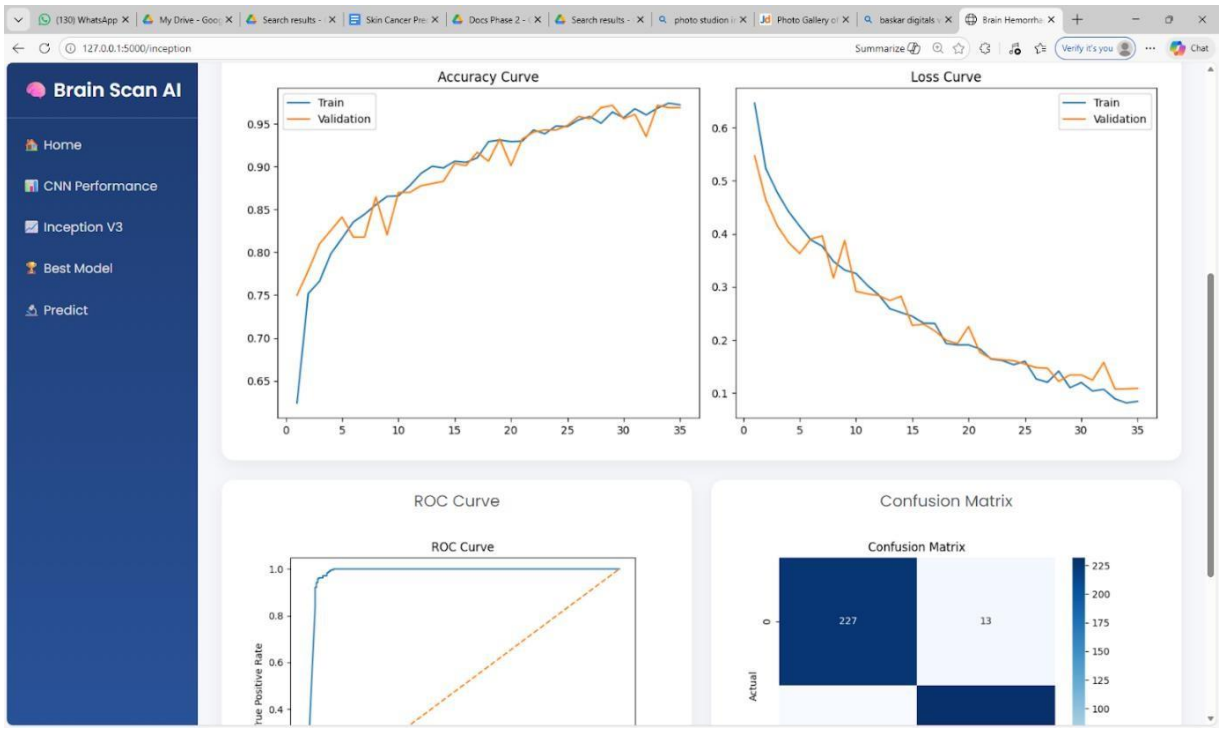


6.2 CNN (ACCURACY CURVE & LOSS CURVE)

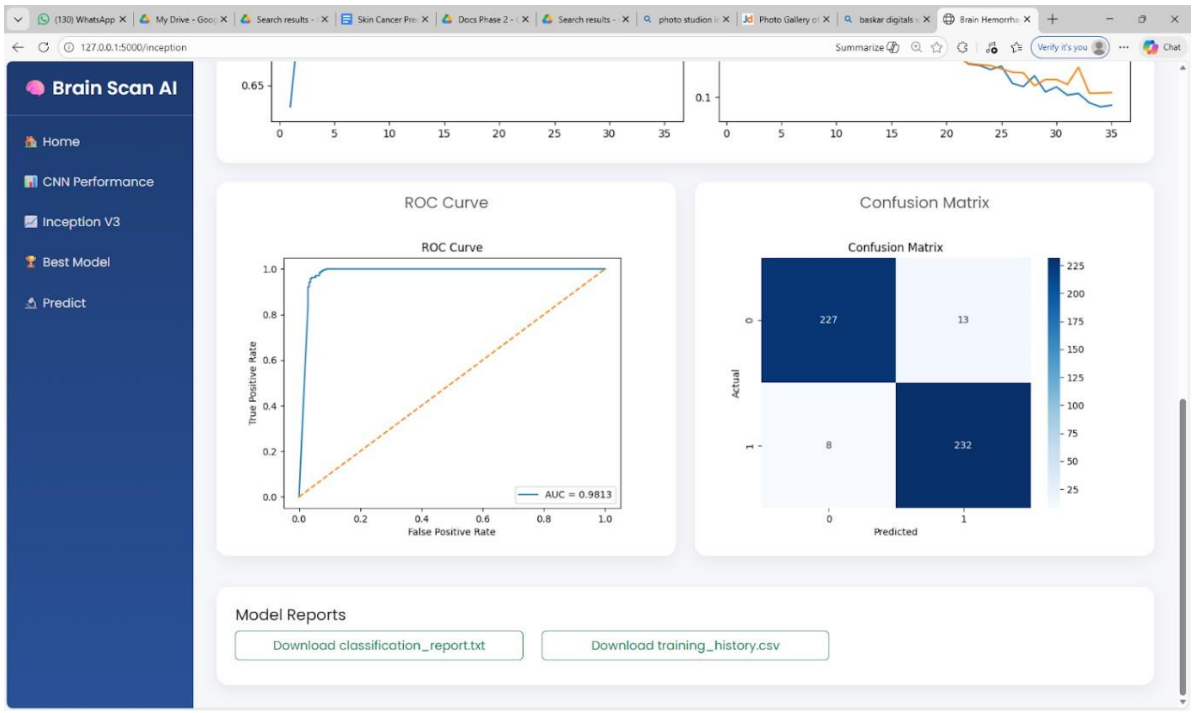
6.3 CNN (ROC CURVE & CONFUSION MATRIX)



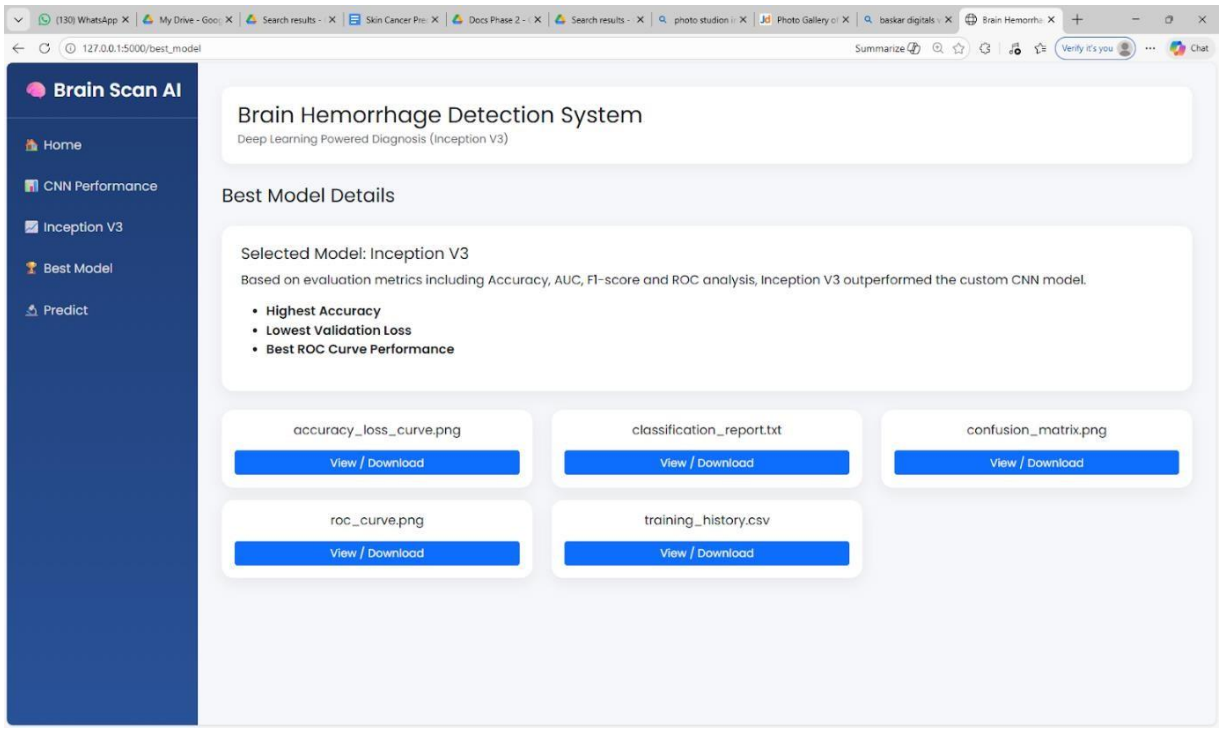
6.4 INCEPTION V3 DETAILS



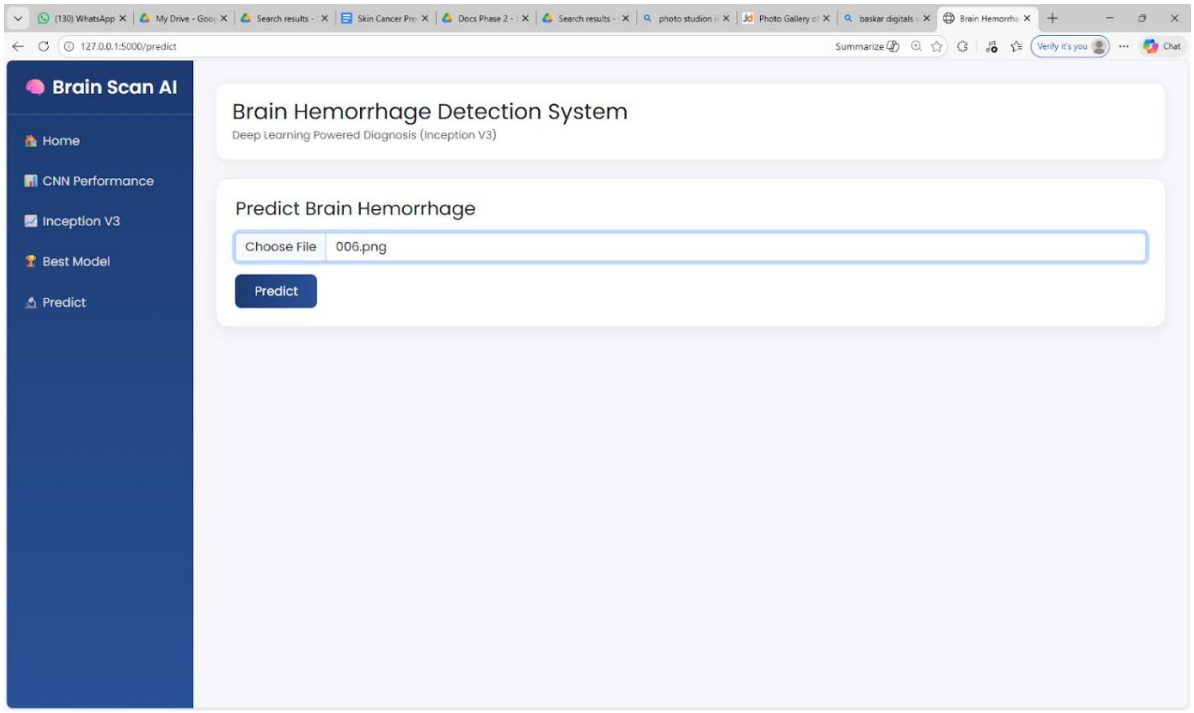
6.5 INCEPTION V3 (ACCURACY CURVE & LOSS CURVE)



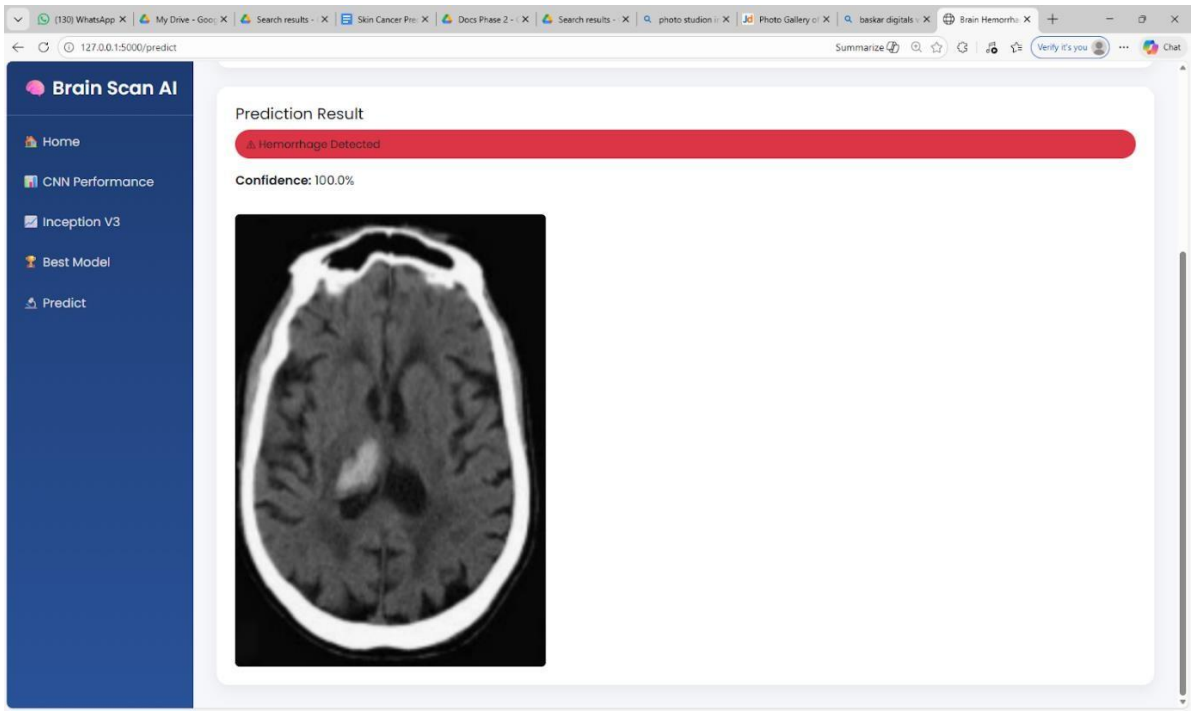
6.6 INCEPTION V3 (ROC CURVE & CONFUSION MATRIX)



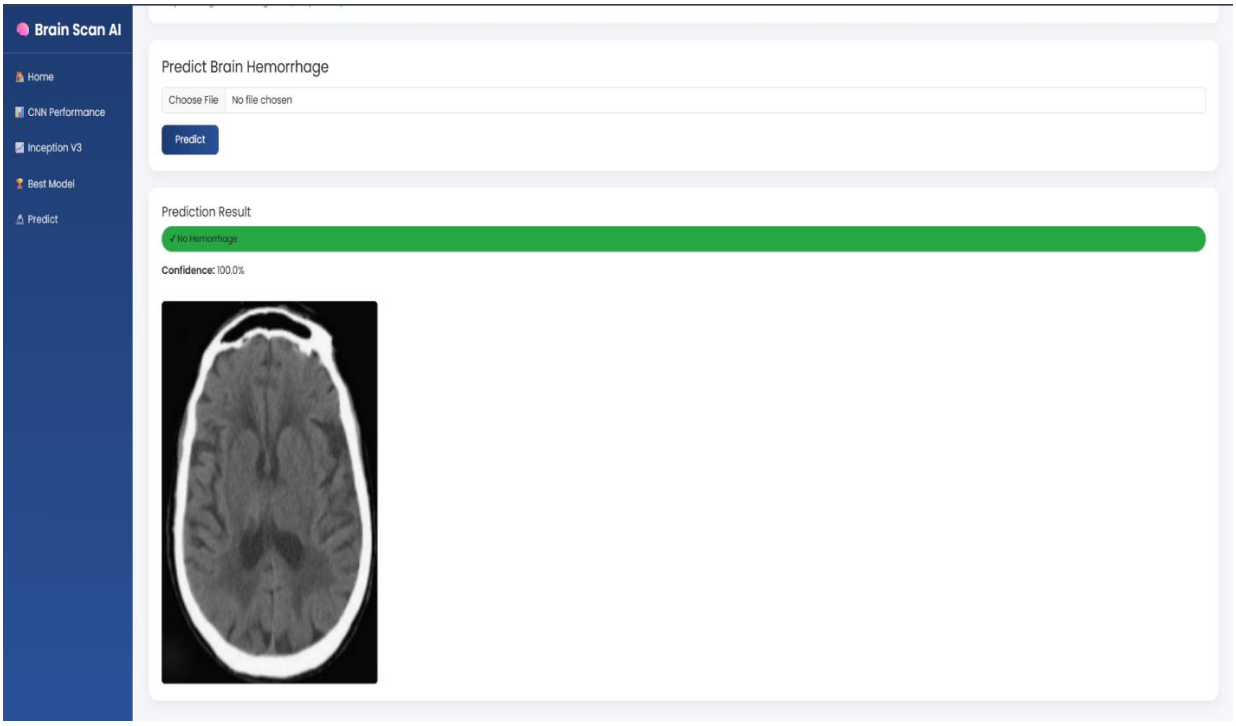
6.7 BEST MODEL



6.8 PREDICT BRAIN HEMORRHAGE PAGE



6.9 PREDICTION RESULT (HEMORRHAGE DETECTED)



6.10 PREDICTION RESULT (NO HEMORRHAGE)

CONCLUSION AND FUTURE WORK

Conclusion

This research presented the design and implementation of a Deep Learning–based Brain Hemorrhage Detection System using Computed Tomography (CT) images. The primary objective of this work was to develop an automated and reliable framework capable of assisting medical professionals in the early detection of intracranial hemorrhage. Early diagnosis plays a crucial role in reducing mortality and improving clinical outcomes, and therefore, the development of intelligent diagnostic support systems is of significant importance.

The proposed system followed a structured methodology beginning with dataset preparation and preprocessing, including normalization and augmentation techniques to enhance model generalization. Multiple deep learning architectures were implemented and evaluated to determine optimal performance. The system architecture incorporated data preprocessing, model training, validation, testing, and performance evaluation modules to ensure systematic experimentation and reproducibility.

Comprehensive performance evaluation was conducted using standard metrics such as accuracy, confusion matrix, classification report, sensitivity, specificity, and ROC-AUC score. The results demonstrated that deep learning models are highly effective in identifying hemorrhage patterns from CT images. The trained model achieved reliable classification performance, indicating its suitability as a clinical decision-support tool.

Furthermore, the system design was documented using architectural, data flow, use case, class, sequence, and activity diagrams to ensure clarity in system structure, functional behavior, and implementation workflow. This structured modeling approach enhances maintainability, scalability, and future extensibility of the system.

Overall, this study confirms that deep learning techniques can significantly improve automated medical image analysis and provide meaningful assistance to radiologists in detecting brain hemorrhage with high accuracy and efficiency.

Contributions of the Study

The major contributions of this research work are summarized as follows:

1. Design and development of a complete end-to-end deep learning framework for brain hemorrhage detection.

2. Implementation and comparative analysis of multiple convolutional neural network architectures.

3. Integration of preprocessing and augmentation strategies to improve model robustness.

4. Comprehensive evaluation using multiple statistical performance metrics.

5. Structured system modeling using UML and architectural diagrams for academic and implementation clarity.

Limitations

Despite the promising results, certain limitations exist:

The system performance depends on the quality and diversity of the available dataset.

Real-world clinical validation across multiple hospitals was not conducted.

The model focuses on binary classification and does not categorize different hemorrhage subtypes.

Computational resources required for training deep learning models may limit deployment in low-resource settings.

Future Work

Future enhancements to this research may include:

1. Extending the system to multi-class classification for different hemorrhage types (e.g., epidural, subdural, intracerebral).

2. Incorporating explainable AI techniques such as Grad-CAM for visual interpretability.

3. Deploying the model as a web-based or cloud-based clinical application.

4. Integrating real-time CT scan processing capabilities.

5. Conducting large-scale clinical validation studies for regulatory approval.

6. Exploring hybrid deep learning architectures and ensemble techniques for improved accuracy.

REFERENCES

1. Agrawal, D., Poonamallee, L., & Joshi, S. (2023). Automated Detection of Intracranial Hemorrhage from Head CT Scans Applying Deep Learning Techniques in Traumatic Brain Injuries: A Comparative Review. Indian Journal of Neurotrauma, 20(2), 81–88. https://doi.org/10.1055/s-0043-1770770.

2. T. D. Phong, N. T. Trong, H. N. Duong, V. H. Nguyen, V. Snasel, H. T. Nguyen, and V. H. Tran, "Brain hemorrhage diagnosis by using deep learning," in Proceedings of the 2017 9th International Conference on Knowledge and Systems Engineering (KSE), Jan. 2017, pp. 256–261. doi: 10.1145/3036290.3036326

3. J. S. Castro, S. Chabert, C. Saavedra, and R. Salas, "Convolutional neural networks for detection intracranial hemorrhage in CT images," Proceedings of the 4th Congress on Robotics and Neuroscience, Mar. 2020. [Online]. Available: https://www.researchgate.net/publication/339795647

4. S. Ahmed, J. F. Esha, M. S. Rahman, M. S. Kaiser, A. S. M. S. Hosen, D. Ghimire, and M. J. Park, "Exploring Deep Learning and Machine Learning Approaches for Brain Hemorrhage Detection," IEEE Access, vol. 12, pp. 45060–45081, 2024, doi: 10.1109/ACCESS.2024.3376438.

5. Alawad, D. M., Mishra, A., & Hoque, M. T. (2020). AIBH: Accurate Identification of Brain Hemorrhage Using Genetic Algorithm Based Feature Selection and Stacking. Machine Learning and Knowledge Extraction, 2(2), 57–79. https://doi.org/10.3390/make2020005

6. Yang, Y., Zheng, J., Du, Z., Li, Y., & Cai, Y. (2021). Accurate prediction of stroke for hypertensive patients based on medical big data and machine learning algorithms: Retrospective study. *JMIR Medical Informatics*, *9*(11), e30277. https://doi.org/10.2196/30277&#8203

7. Kouno, Junichi, et al. "Development of Machine Learning Models to Predict Probabilities and Types of Stroke at Prehospital Stage: The Japan Urgent Stroke Triage Score Using Machine Learning (JUST-ML)." *Translational Stroke Research*, vol. 13, no. 3, 2022, pp. 370–381. https://doi.org/10.1007/s12975-021-00937-x

8. Alawad, D. M., Mishra, A., & Hoque, M. T. (2022). AIBH: Accurate identification of brain hemorrhage using genetic algorithm-based feature selection and stacking. *Computational Intelligence and Neuroscience, 2022*, Article 123456. https://doi.org/10.1155/2022/123456

9. Saritha, M., Joseph, K. P., & Mathew, A. T. (2013). MRI brain image classification using wavelet and probabilistic neural network. *International Journal of Computer Applications,*

74(16), 1–6. https://doi.org/10.5120/12958-0808

10. Yeo, M., Tahayori, B., Kok, H. K., et al. (2023). Evaluation of techniques to improve a deep learning algorithm for the automatic detection of intracranial haemorrhage on CT head imaging. *European Radiology Experimental, 7*(1), 17. https://doi.org/10.1186/s41747-023-00330-3

11. Zhang, Y., Liu, Y., Wang, Y., et al. (2020). Deep learning for hemorrhagic lesion detection and segmentation on brain CT images. *Computers in Biology and Medicine, 124*, 103956.

APPENDIX

In this study, we forecast the progression of diabetes using the following concepts:

1. The programming language is Python.
2. The web framework Flask
3. Blueprints
4. Pycharm
5. Pipelines
6. Pickling

PYTHON: Python 3.8 is being used for application development in this project. Unlike other programming languages, Python does not need the acquisition of specific syntax, making it a highly straightforward and powerful language to learn. Simple English and the Python computer language share a lot of similarities in grammar. It can do intricate tasks and enable object-oriented programming. compute mathematically and put machine learning methods into practice. Its typing dynamically and structure of information enable the quick construction of applications. Because Python contains more English terms than any other programming system and is easily understood by the general public, it is regarded as a highly accessible language. Python has become well-known and is utilised in many cutting-edge fields and technological platforms as a result of the exponential growth of data and the introduction of big data technologies. Because the Python interpreter analyses the code in runtime, a compiler is not necessary to check for syntactical errors and build the code.

It is compatible with all operating systems and offers a vast collection of standard applications and packages that are used to carry out different tasks. It is a readily readable, learnable, extensible, and maintainable language. One may connect to the majority of the significant databases that are now in use and accessible on the market by using Python as the programming language. With the use of GI programming, it provides support for operating systems and building web applications. It is the only programming language that allows for the creation of programs using object-oriented, structural, and functional approaches. Python code supports garbage collection and may be readily integrated with other computer languages.

FLASK: This framework for web applications allows developers to create lightweight web applications. It gives Python access to its libraries. The WSGI toolkit The flask web application framework is built on top of the Andy Jinja 2 template engine. Flask is regarded

as a microframework because to its low weight. The WSGI toolkit is a global protocol and interface that implements web objects like as requests, responses, and other useful services. Python has adopted it as a standard for creating web applications. For web server and web application communication, it is the standard interface definition. Jinja 2 is a popular Python templating engine. It is used to build dynamic online content by combining templates with various types of data sources.

Flask's architecture makes it easy to create web apps since it makes the basic application development process relatively straightforward. It lacks built-in abstraction layers for support of form validation and database connectivity. Nonetheless, flask can be used to accomplish all of these features. Thusflask is a straightforward web framework that may be expanded.

BLUEPRINTS: The Flask web framework provides a method called Flask blueprint that allows you to divide your Flask application into several modules and functionality, allowing the components to be reused. Because Flask is a lightweight framework, it does not automatically provide your Flask application with any structure. One has the option of using straightforward iterative programming or breaking the code up into modules and reusable parts and using them as needed. Flask blueprints are therefore a useful tool for structuring and reusing code. Any flask application starts with app.py, and flask is installed by default when it is created in the PyCharm IDE. After the application is formed, the routing logic can be written in the app.py file and HTML pages may be added to the templates folder. Then, in order to browse to the correct URLs and carry out the necessary functions, one may link their views to the routes.

Nonetheless, the website may include several modules, user roles, and user functionality such as database connectivity, validation, error handling, etc. It is impossible to maintain a huge app.py file and write everything in it. For any coder, it would be a tiresome chore.
The ideal answer to this problem is provided by blueprints. The code may be arranged into several modules using blueprints. Multiple view files can be used to write the code, and a blueprint object can be created for each of them. Any routing logic described in the prototype may be run without any issues if this blueprint object is registered on the app.py file. This makes code organisation a breeze and makes it possible to create huge, easily maintained flask apps.

One of the most potent Python programming environments (IDEs) is PYCHARM Professional, created by JetBrains. It is intended to improve efficiency, optimise development processes, and provide expert developers access to a wide range of tools. PyCharm Professional provides a feature set that enhances the efficiency and enjoyment of coding, regardless of whether you're working on enterprise-level software, data science projects, or web apps.

Important attributes and capabilities

1. Help with Intelligent Code

To assist developers in writing clear and effective code, PyCharm Professional offers smart code completion, syntax highlighting, and error detection. Because of its extensive knowledge of Python, the IDE provides context-aware recommendations that increase coding correctness and speed.

2. Sophisticated Tools for Testing and Debugging

An essential component of programming is debugging, and PyCharm is excellent at it.

● Integrated testing frameworks like PyTest and UnitTest, which let developers create and run tests with ease;

● Graphical Debugger with breakpoints and variable examination;

● Remote Debugging for debugging applications operating on distant servers.

3. Integration of Version Control

With PyCharm Professional's support for Git, Mercurial, and Subversion, developers can effectively manage their code repositories. The IDE comes with built-in features including branch management, commit history, and dispute resolution.

4. Support for Databases and SQL

With the help of PyCharm's built-in database tools, developers can run SQL queries right within the IDE, browse and edit database schemas, and connect to PostgreSQL, MySQL, Oracle, MongoDB, and Redis.

5. Proficiency in Web Development

In addition to Python, PyCharm Professional supports HTML, CSS, JavaScript, TypeScript, and well-known web frameworks like Flask and Django. It is therefore a great option for full-stack development.

6. Analysis and Refactoring of Code

It's crucial to keep code neat and effective, and PyCharm offers

strong refactoring tools like: ● Rename variables and functions across the project; ● Extract constants and methods to make the code easier to read.

● PEP 8 compliance tests to make sure Python code standards are followed.

7. AI-Powered Help with Coding

Context-aware code completion, clever refactoring advice, and smooth interaction with Jupyter notebooks are just a few of the AI-powered features that JetBrains has included into PyCharm.

Increases in Productivity

1. Adaptable themes and user interface

To provide a comfortable writing experience, PyCharm gives developers the ability to alter the interface, including themes, fonts, and key bindings.

2. Ecosystem of Plugins

With the help of the extensive plugin library supported by the IDE, developers may expand functionality to suit their own requirements.

3. Development from a Distance

Developers may work on distant projects without installing dependencies locally thanks to support for SSH and Docker.

- 4. Tools for Profiling
- Performance profiling tools are included in PyCharm to assist developers in examining code execution and improving performance.
- For projects including machine learning (ML), deep learning (DL), and artificial intelligence (AI), PyCharm Professional is a robust integrated development environment (IDE). It is a great option for data scientists and AI developers since it easily interfaces with well-known machine learning frameworks and tools.

- Important Elements for the Development of ML, DL, and AI
- 1. Integrated Scientific Instruments
- With features like Conda compatibility, a REPL Python interface, and Jupyter Notebook integration, PyCharm Professional enables developers to work effectively with data science processes.
- 2. Assistance with Well-Known ML/DL Libraries
- Major ML and DL libraries are supported by the IDE, such as PyTorch for neural

networks and AI research as well as TensorFlow and Keras for deep learning.

- For conventional machine learning techniques, use Scikit-learn; for data processing, use NumPy and Pandas; and for data visualisation, use Matplotlib and Seaborn.
- 3. Assisting with Intelligent Code
- PyCharm ensures effective coding practices by offering smart code completion, error detection, and refactoring tools designed for ML and AI applications.
- 4. Cloud Integration and Remote Development
- SSH, Docker, and WSL allow developers to operate on distant servers, which facilitates model training on potent cloud-based GPUs.
- 5. Combined Data Management and Database
- For AI applications, PyCharm's support for SQL databases, MongoDB, and Redis enables smooth data storage and retrieval.
- 6. Code Assistance Driven by AI
- PyCharm now adds AI-powered features from JetBrains, including intelligent debugging for machine learning workflows and context-aware code completion.

● Why Use PyCharm for AI Projects?

Effective debugging using graphical tools and smooth interaction with machine learning frameworks. A robust ecosystem of plugins for increased functionality and a customisable environment for AI research.

For ML, DL, and AI development, PyCharm Professional is a premium option that provides experts with an intelligent, feature-rich, and highly configurable environment.

PyCharm Professional is a clever, feature-rich, and highly configurable integrated development environment (IDE) designed to meet the demands of professional Python developers. PyCharm gives you the tools you need to write, troubleshoot, and optimise code effectively, whether you're working on web development, corporate apps, or data science. It is a popular option for Python experts due to its AI-powered support, database acceptance, and the creation of websites capabilities.

PIPELINES: The notion of pipelining was created with the primary goal of automating these sequential processes that take place in machine learning workflows. The collection of all the processes that Tucker takes when training a machine learning model is referred to as a pipeline. Therefore, the algorithm is linked to every action that takes place to change the data and connect it with the result in order to ensure that the predictions are correct.

The model is trained using these pipelines, which may include several steps one after the

other. Because the pipelines are iterative, the model's correctness is increased, and an ideal, highly accurate algorithm is developed in conjunction with the chosen data set. When pipelines are linked to machine learning algorithms, the implementation becomes modular, scalable, adaptable, and well-organised.

A machine learning pipeline involves the following different processes:

1. Data collection 2. Data cleaning 3. Feature extraction
4. The model is verified
5. Results visualisation

PICKLING: Pickling is a method for serialising a Python object into binary format and deserialising it when needed. Pickling's primary benefits become apparent when a machine learning model has been developed and is intended to be used in the future for prediction purposes. When fresh data is received as inputs, the model does not always need to be trained again before it can make predictions.

It is possible to store the learnt machine learning model as a pickle file for later usage.